BusXTrack

Smart College Bus Tracking and ETA System

Project Design, GUI Prototype and MySQL Database Report
Mid-Term I Project-Based Evaluation

Department of Computer Engineering
Academic Year 2026

Certificate

This is to certify that the project report titled BusXTrack Smart College Bus Tracking and ETA System has been prepared as part of the Project-Based Evaluation for the third semester Java mini-project. The work describes the design, Java Swing prototype and MySQL database prepared by the student group.

Faculty signature: _____

Date: _____ Place: _____

Declaration

We declare that this report presents the design and prototype of BusXTrack, a schedule-based college bus tracking and ETA simulation system. The report describes the scope honestly: the current prototype does not use GPS, Google Maps or external APIs. It uses Java Swing, JDBC, MySQL and fixed timetable data.

Student signatures: _____

Group members: _____

Acknowledgement

We thank our department and project guide for providing the opportunity to work on a Java and database mini-project. The project helped us connect object-oriented programming concepts with a graphical user interface, SQL tables and JDBC database access.

We also acknowledge the value of testing the prototype through small examples such as bus BX101 and the Market Road stop. These examples make the schedule calculation easy to demonstrate and explain.

Abstract

BusXTrack is a desktop application that organizes college bus information for two users. The administrator maintains driver, route, stop, bus and timetable records. The student searches a bus number or stop name and views the assigned route, driver, expected arrival and current schedule status.

The project uses a transparent simulation model. The application reads the current time from the computer and compares it with a timetable departure. It adds the configured minutes from the route start to calculate an expected stop time. This makes the project useful as a learning prototype while clearly separating it from a real GPS tracking product.

Keywords: Java Swing, MySQL, JDBC, college transport, timetable, ETA simulation, desktop application

1 Executive Summary

BusXTrack is a beginner-friendly Java Swing and MySQL application for a college bus office. It helps an administrator maintain buses, drivers, routes, stops and timetables. A student can search for a bus or a stop and see its route, driver details and schedule-based simulated ETA.

The prototype does not use GPS, Google Maps or external APIs. It reads the current time from the computer running the application. It then compares that time with a timetable departure time and the configured minutes of each stop. For example, if a bus departs at 08:00 and Market Road is 15 minutes from the route start, the expected arrival is 08:15. At 08:10, the interface shows approximately 5 minutes remaining.

Item	Project decision
Domain area	College transportation and timetable management
Technology	Java, Swing, JDBC and MySQL
Innovation	Simple schedule-based ETA simulation for a college bus workflow
Expected outcome	A working GUI prototype connected to a normalized MySQL database
Area of utilization	College transport office and students checking bus schedules

2 Problem Statement, Objectives and Scope

Students often do not know which college bus serves a particular stop, when it is expected, or which driver is assigned. Manual timetable sheets can be difficult to search and update. BusXTrack provides one simple desktop interface for maintaining this information and presenting it in a student-friendly way.

Objectives

- Store bus, driver, route, stop and timetable information in MySQL.
- Provide separate admin and student workflows in Java Swing.
- Allow a student to search by bus number or stop name.
- Calculate an estimated arrival time from a fixed schedule and the current system time.
- Demonstrate database integration through JDBC and simple DAO classes.

Scope

- Included: local desktop prototype, CRUD forms, search, route view, driver details, timetable and simulated ETA.
- Excluded: live GPS, maps, mobile deployment, notifications, online authentication and real-time vehicle sensors.
- The prototype is designed for one college and a small sample dataset.

3 Requirements and Feasibility

Functional Requirements

User	Requirement	Prototype screen
Admin	Login with username and password	LoginFrame
Admin	Add, edit and delete buses	BusManagementFrame
Admin	Manage drivers	DriverManagementFrame
Admin	Manage routes and stops	RouteManagementFrame and StopManagementFrame
Admin	Manage departure timetable	TimetableManagementFrame
Student	Search by bus number	SearchBusFrame
Student	Search by stop name	SearchStopFrame
Student	View route and driver details	RouteViewFrame and BusDetailsFrame
Student	View simulated ETA and bus status	Search result tables

Non-functional Requirements

- The interface should be simple enough for a student to use without training.
- Validation should stop empty form submissions and show a message to the user.
- The code should remain readable and easy to explain in a viva.
- The application should work on a local computer with Java, MySQL and the JDBC connector.

Feasibility Study

Type	Finding
Technical	Feasible because Java Swing, JDBC and MySQL are available and suitable for a desktop prototype.
Operational	Feasible for a college transport office because the admin maintains the timetable and students only search information.
Economic	Low cost because the project uses free/open-source development tools and local software.
Schedule	Feasible for a semester mini-project because the scope is limited to fixed schedules and basic CRUD.

4 System Architecture and Module Design

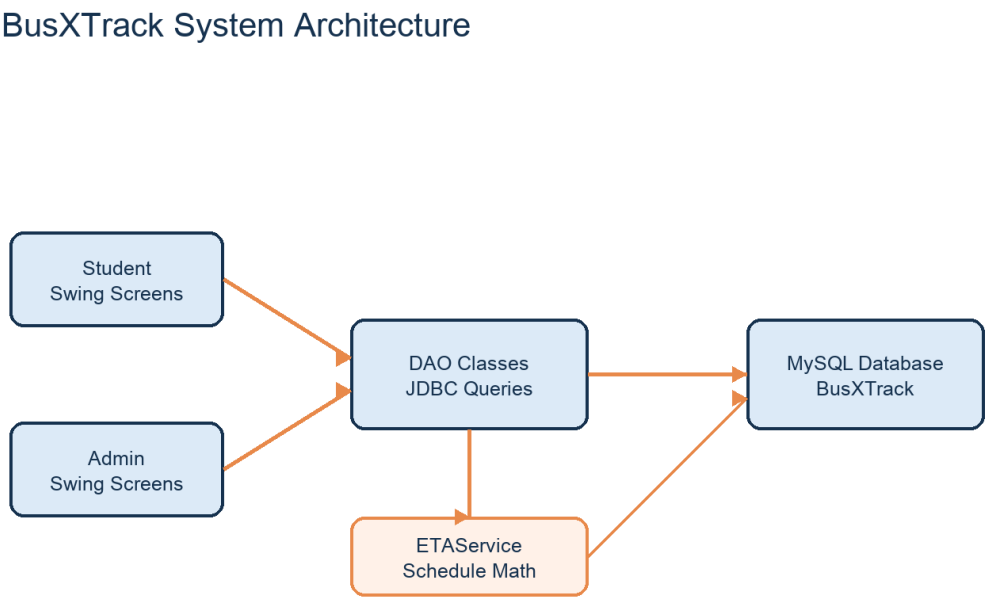


Figure 1. Layered architecture used in the prototype

The user interface contains Swing frames. DAO classes contain the database operations and use DBConnection to create a JDBC connection. Model classes represent records such as Bus and Driver. ETAService contains the schedule calculation so that the formula can be tested and explained separately from the GUI.

Module	Main classes	Responsibility
Models	Admin, Bus, Driver, Route, Stop, Timetable	Store data for one entity using simple fields and getters/setters.
Database	DBConnection	Create a connection to the local busxtrack MySQL database.
DAO	AdminDAO, BusDAO, DriverDAO, RouteDAO, StopDAO, TimetableDAO	Run prepared SQL queries for login, CRUD and search.
Service	ETAService	Compare current time with departure and stop minutes.

GUI	LoginFrame, AdminDashboardFrame, StudentHomeFrame and child frames	Collect input, show tables and navigate between workflows.

5 UML and System Design Diagrams

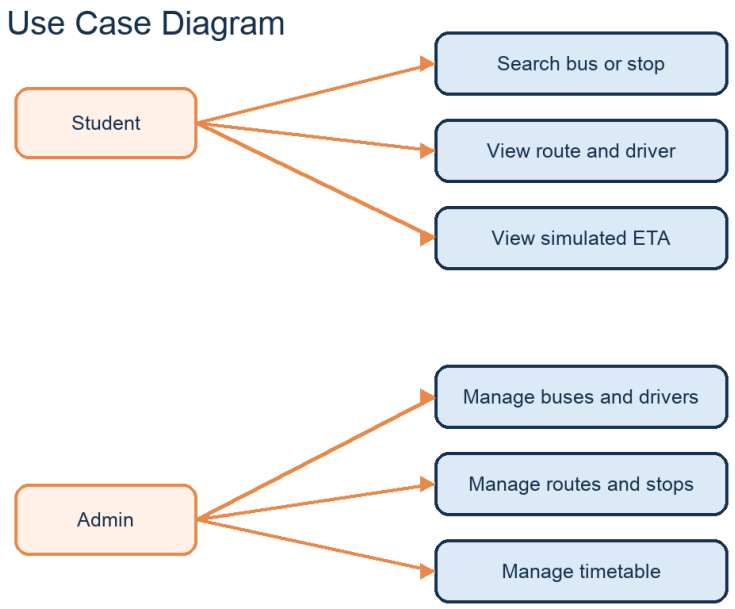


Figure 2. Main user actions

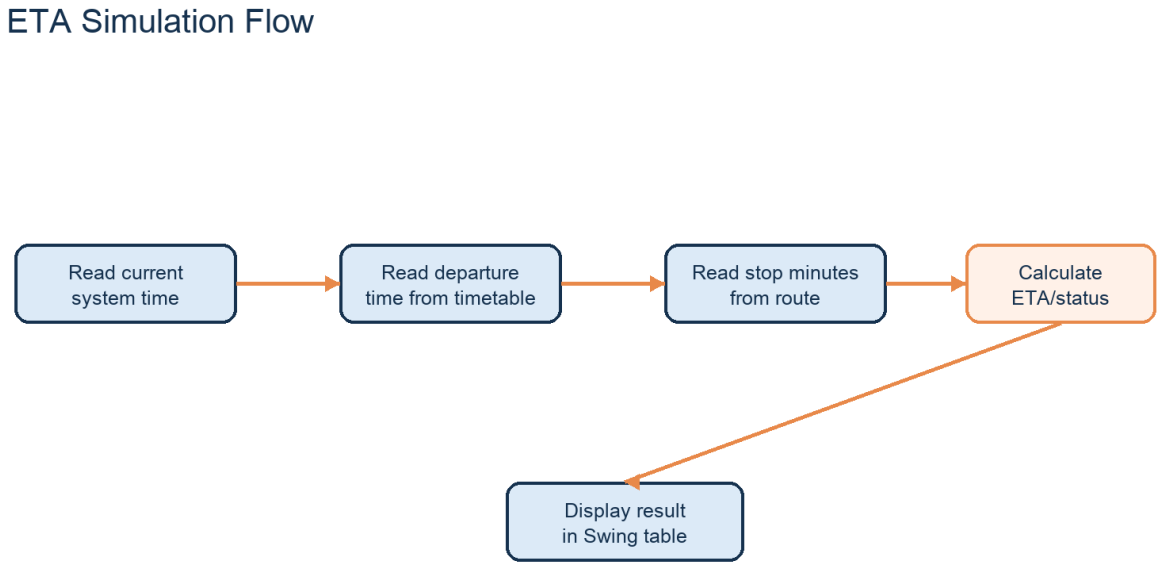


Figure 3. Schedule-based ETA calculation flow

The ETA is calculated as: expected stop time = timetable departure time + minutes_from_start. Remaining time = expected stop time - current system time. The application also compares the current time with the first and last stop to show NOT STARTED, RUNNING or COMPLETED.

6 Java Swing GUI Prototype

Screen	What can be demonstrated
LoginFrame	Admin login with validation; student can continue without admin login.
AdminDashboardFrame	Buttons open bus, driver, route, stop and timetable management.
BusManagementFrame	Form and table for bus number, route, driver and capacity.
DriverManagementFrame	Form and table for driver name, phone and license.
RouteManagementFrame / StopManagementFrame	Create routes and attach ordered stops with minutes from start.
TimetableManagementFrame	Assign a departure time and trip type to a bus.
StudentHomeFrame	Student workflow entry point.
SearchBusFrame	Shows trip, departure, status, next stop and ETA.
SearchStopFrame	Finds buses that include a selected stop and shows expected arrival.
RouteViewFrame / BusDetailsFrame	Shows route stops and assigned driver information.

Navigation and Demonstration Workflow

Application start -> Login screen -> choose Continue as Student or Admin Login. Student -> Search Bus / Search Stop / View Routes -> select or enter data -> view schedule-based result. Admin -> Dashboard -> choose a management module -> fill form -> Add or Update -> refresh table -> return to dashboard.

Validation

- Login checks that username and password are not empty.
- Search screens check that the search field has a value before querying.
- Details screens require a successful search before opening bus details.
- Database exceptions are displayed in a Swing message dialog.
- The database uses NOT NULL, UNIQUE, PRIMARY KEY and FOREIGN KEY constraints.

7 MySQL Database Design

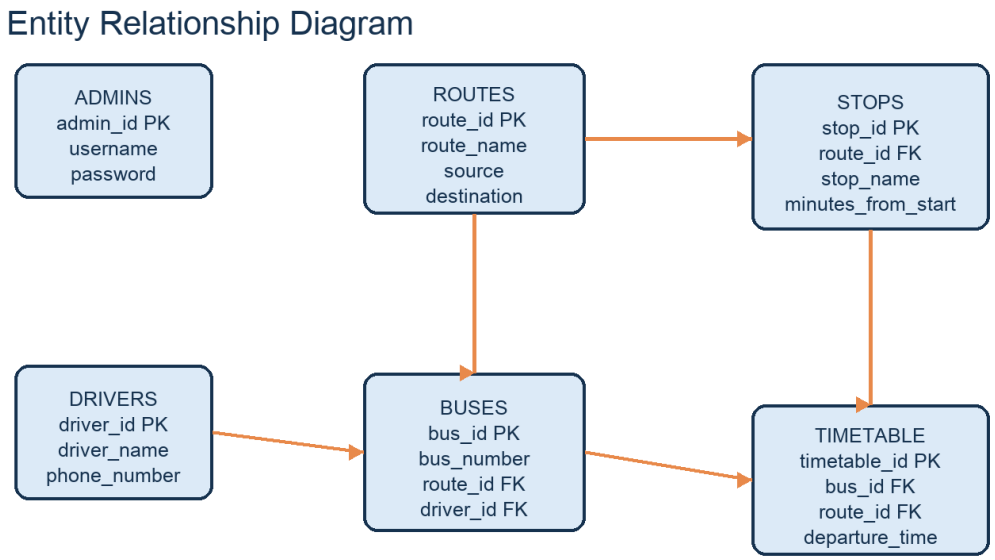


Figure 4. Relational entities and key relationships

Table	Primary key	Important attributes	Foreign keys
admins	admin_id	username, password	None
drivers	driver_id	driver_name, phone_number, license_number	None
routes	route_id	route_name, source, destination	None
stops	stop_id	stop_name, stop_order, minutes_from_start	route_id -> routes
buses	bus_id	bus_number, capacity	route_id -> routes; driver_id -> drivers
timetable	timetable_id	departure_time, trip_type	bus_id -> buses; route_id -> routes

Relationships

- One route has many stops.
- One route can be assigned to many buses.
- One driver can be assigned to a bus record.
- One bus can have multiple timetable records, such as morning and evening trips.
- Foreign keys keep related records consistent and ON DELETE CASCADE removes dependent stops/timetable rows where configured.

Normalization

The schema follows basic normalization. In first normal form, each field stores one atomic value. In second normal form, non-key attributes describe the complete primary key of their table. In third normal form, driver data, route data and stop data are separated instead of being repeated inside the bus or timetable row. Foreign keys connect the entities without storing duplicate descriptions.

Sample SQL Queries

Purpose	Query idea

Find a bus	SELECT * FROM buses WHERE bus_number = ?
List stops	SELECT * FROM stops WHERE route_id = ? ORDER BY stop_order
Find buses for a stop	JOIN stops, buses and routes using route_id
View a bus schedule	SELECT * FROM timetable WHERE bus_id = ?
Show driver details	JOIN buses and drivers using driver_id

The complete executable schema and sample records are in database/busxtrack.sql. The submission folder also includes a readable SQL appendix with demonstration queries.

8 Integration and Current Progress

Component	Current status
Java source structure	Complete prototype source under src/busxtrack with model, DAO, service and UI packages.
Swing navigation	Complete for login, admin dashboard and student search workflow.
MySQL schema	Complete schema with keys, constraints and sample data.
JDBC integration	Implemented through DBConnection and DAO classes; local MySQL credentials may need editing.
ETA logic	Implemented in ETAService using LocalTime and stop minutes.
Documentation	This report, diagrams, SQL appendix, presentation and viva/demo notes.

Limitations and Future Scope

- The current version simulates movement from a timetable and does not know the physical GPS position of a bus.
- The database is local and intended for demonstration on one computer.
- Possible future work includes GPS hardware, a mobile app, notifications, student accounts and an online server.

9 Development Roadmap

Phase	Work	Status
1	Requirements, scope and database design	Completed
2	Create model classes and MySQL schema	Completed
3	Build DAO classes and JDBC connection	Completed
4	Build login, admin and student Swing screens	Completed prototype
5	Connect schedule data to ETAService	Completed prototype
6	Testing, report, diagrams and presentation	Prepared for mid-term
7	Future: improve validation and add live location	Post-mid-term scope

10 Conclusion

BusXTrack demonstrates a complete small desktop application flow. The administrator maintains transport data, the student searches the available information, and the system calculates a transparent ETA from stored schedule values and the current computer time. This scope is realistic for a third-semester Java project because every part can be traced from a Swing input field to a JDBC query and a MySQL table.

11 Background of the Project

College transport information is often maintained using printed notices, spreadsheets or informal messages. These methods can work for a small number of buses, but students still need to ask which bus serves a stop and whether the bus is expected before or after a particular time.

BusXTrack treats the timetable as the main source of truth. A route contains an ordered list of stops. Each stop stores the approximate number of minutes from the route start. A bus is assigned to a route and a driver, and the timetable stores one or more departures for that bus.

This design gives the project a clear academic boundary. The team can demonstrate database design, Swing components, object-oriented classes and a calculation service without pretending that a desktop prototype has live location data.

The project is also a useful example of how a real-world problem can be reduced to a manageable semester scope. A real transportation product may contain mobile devices, location services, network communication and notification systems. This prototype keeps the user-visible question, which bus should I take and when should I expect it, while implementing the data and calculation part with tools already taught in the course.

12 Existing System and Proposed System

In the existing manual approach, route and driver information may be written in separate registers. A student must find the correct sheet and mentally compare the current time with the departure time. Updates can be missed when a bus, driver or route changes.

The proposed system gives the administrator forms for maintaining the records and gives the student search screens for reading them. The same database is used by both workflows, which reduces repeated manual entry. The ETA result is calculated from data that can be traced back to the timetable and stop records.

The proposed version is intentionally small. It is a desktop prototype for a single college and does not attempt to solve traffic prediction, GPS accuracy or internet-scale deployment.

The proposed workflow also improves traceability. If a student sees that Market Road is expected at 08:15, the result can be explained using the Route A stop row and the BX101 timetable row. This is better for a viva because the displayed result is connected to a stored record rather than presented as an unexplained value.

13 Detailed Objectives

The first objective is to model the college transport domain using simple Java classes. Bus, Driver, Route, Stop and Timetable are represented separately so that each class has a clear purpose.

The second objective is to connect the Java program to MySQL using JDBC. DAO classes perform the database operations and return data that can be displayed in Swing tables.

The third objective is to build a usable student workflow. A user can search by a bus number or stop name and can open a route or driver detail view without needing to understand the database.

The final objective is to demonstrate an explainable ETA formula. The result is deliberately based on stored schedule values and current computer time rather than an invisible or external service.

Each objective has a corresponding evidence item in the project. Java classes demonstrate object-oriented modelling, DAO classes demonstrate database access, Swing frames demonstrate interaction, the SQL script demonstrates relational design and ETAService demonstrates the calculation. This mapping helps the evaluator connect the report to the live prototype.

14 Functional Requirements in Detail

The login requirement allows an administrator to enter a username and password. The student path remains available through a separate Continue as Student button because students do not need CRUD access.

The administration requirement includes create, read, update and delete operations for drivers, routes, stops, buses and timetables. Tables display the stored records and selecting a row fills the form for editing.

The student requirement includes bus search, stop search, route viewing and bus detail viewing. The search screens use text fields, buttons and JTable components to display multiple results.

The calculation requirement includes expected stop time, ETA text and an overall status. The system also handles blank search fields with message dialogs.

The requirements are intentionally written as user-visible actions. A requirement is considered useful only when it can be demonstrated through a screen, a database row or a calculation. This prevents the documentation from claiming features such as GPS or mobile notifications that are outside the current implementation.

15 Non-functional Requirements

Usability: labels and buttons should make the main actions obvious to a beginner. The prototype uses separate frames instead of a complicated navigation framework.

Maintainability: the code is separated into model, DAO, service and UI packages. Methods are short enough for a third-semester student to trace during a viva.

Reliability: database failures are caught as SQL exceptions and shown through JOptionPane messages. Required values are checked before insert or update operations.

Portability: the application is designed to run locally with a Java SDK, the MySQL Connector/J jar and a MySQL server on the same computer.

The prototype also uses simple naming and direct control flow so that students can read it during a practical examination. It avoids frameworks, threads and complex patterns. This is a design decision, not a missing feature: the project prioritizes clarity and successful demonstration at third-semester level.

16 Feasibility Details

Technical feasibility is high because Swing and JDBC are part of the Java learning path and MySQL is a common relational database for student projects. The connector jar is stored in the project lib folder.

Operational feasibility is suitable for a college transport office. An administrator maintains the schedule and students read the published information. The system does not require specialist hardware for the prototype.

Economic feasibility is suitable for a mini-project because the prototype uses local software and a small database. Future GPS hardware would be a separate project phase.

Time feasibility is supported by limiting the first version to fixed routes, simple forms and a transparent schedule calculation.

The most important feasibility assumption is that the college transport data is maintained by an admin. The system does not attempt to discover routes automatically. Once this assumption is accepted, the database tables and the GUI forms provide all the information required for the demonstration.

17 Object-oriented Design

The model classes use encapsulation through private fields and public getters and setters. For example, a Bus object carries a bus id, bus number, route id, driver id and capacity.

The DAO classes group operations by entity. BusDAO manages bus queries, DriverDAO manages driver queries and so on. This is a simple separation of responsibilities rather than a complex enterprise pattern.

The ETAService class keeps time calculations out of the Swing frames. The UI asks the service for status, expected time and ETA text. This makes the main formula easier to explain and change.

The Main class only starts LoginFrame. This keeps application startup separate from the details of each screen.

This arrangement demonstrates composition of a small system from understandable classes. The screen does not need to know how SQL is written, and the ETA service does not need to know how a button is laid out. The boundaries are simple enough to explain with a class diagram and a short code walk-through.

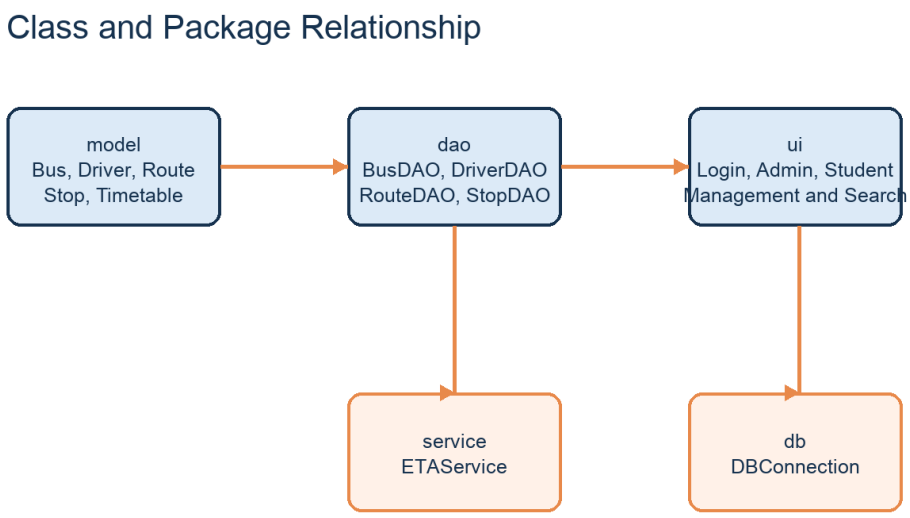


Figure 5. Relationship between the main Java packages

18 Package and Class Design

The busxtrack.model package contains Admin, Bus, Driver, Route, Stop and Timetable. These classes represent records returned from MySQL.

The busxtrack.dao package contains AdminDAO, BusDAO, DriverDAO, RouteDAO, StopDAO and TimetableDAO. Each DAO uses SQL statements and JDBC result sets.

The busxtrack.ui package contains the login, dashboards, management screens and student search screens. ComboItem is a small helper used to show names while keeping the related id in combo boxes.

The busxtrack.db package contains DBConnection. The busxtrack.service package contains ETAService. The structure is small enough to be understood without dependency injection or frameworks.

The package names also make navigation easier in IntelliJ IDEA. A student can open the ui package to find screens, the dao package to find SQL-related methods and the service package to find the ETA formula. This organization reduces the chance of putting all code in one very large class.

19 User Interface Design

The LoginFrame is the first screen. It contains username and password fields, an Admin Login button and a Continue as Student button.

The AdminDashboardFrame contains one button for each management module. The student workflow begins at StudentHomeFrame and contains buttons for bus search, stop search and route viewing.

Management frames use a form on one side and a JTable on the other side. This arrangement lets the user enter a record, see the existing data and select a row for update or deletion.

Search frames use a simple input area and a result table. This keeps the interface suitable for a prototype demonstration rather than adding unnecessary screens.

The interface is deliberately plain because it is a prototype. The important evidence is the presence of working controls, validation, tables and navigation. During the demonstration, the evaluator can see the data entry path and then verify the result against MySQL and the timetable values.

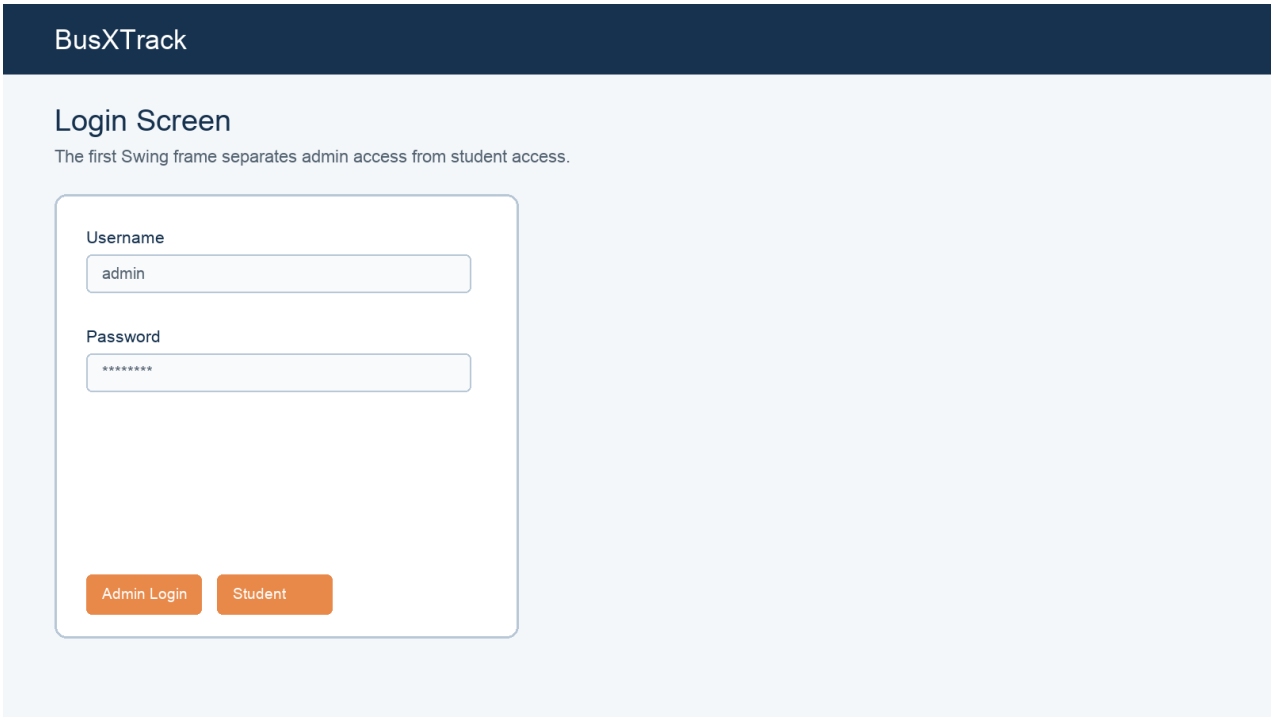


Figure 6. Login screen mockup used to explain the Swing workflow

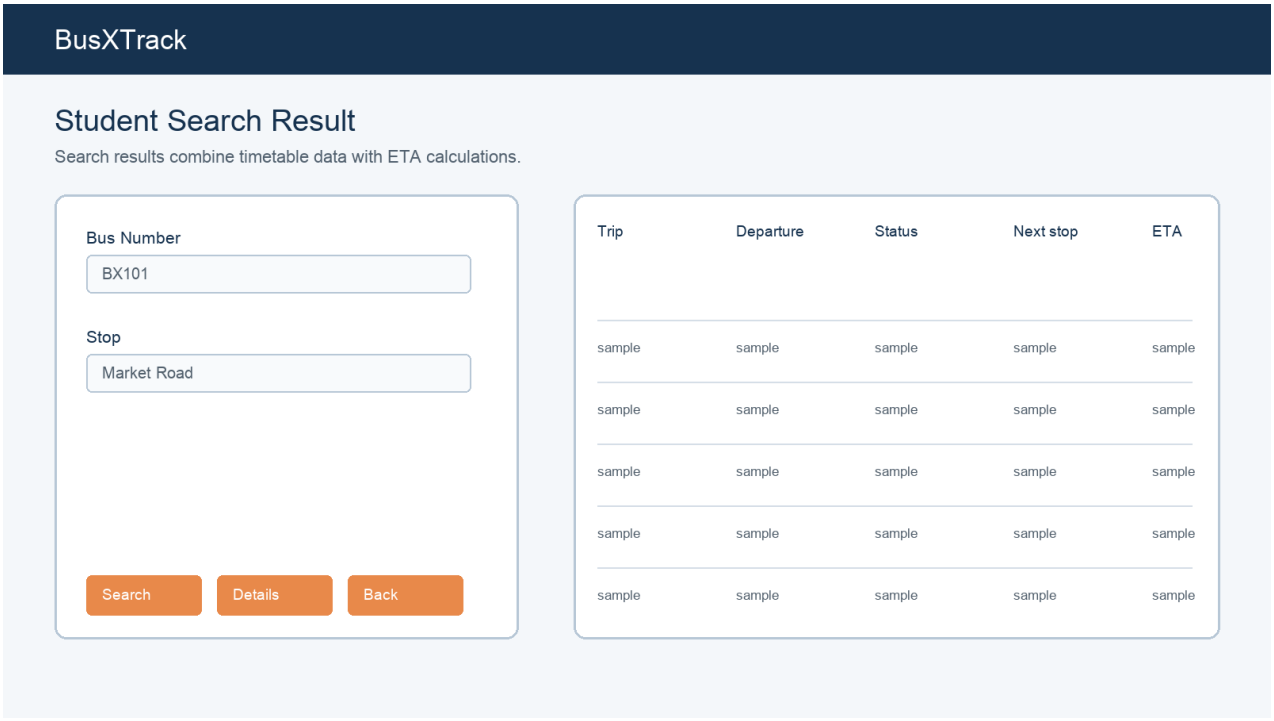


Figure 7. Student search result mockup

20 Navigation and Interaction

The application uses JFrame navigation. When a button opens the next screen, the current frame is disposed so that the user does not end up with many open windows.

The Back button returns to the previous logical screen. Logout returns to LoginFrame. Search results can open a detail screen when the user has first selected or successfully searched for a bus.

The UI displays message dialogs for empty inputs, invalid numbers, invalid time formats, missing records and database errors. These dialogs are part of the prototype validation demonstration.

The most reliable live demo is to use the preloaded records first and only add new data if the examiner asks for CRUD.

A good demonstration should show both a successful path and one validation path. For example, search BX101 successfully, then clear the field and press Search to show the validation dialog. This proves that the interface accepts user interaction rather than being a collection of static screenshots.

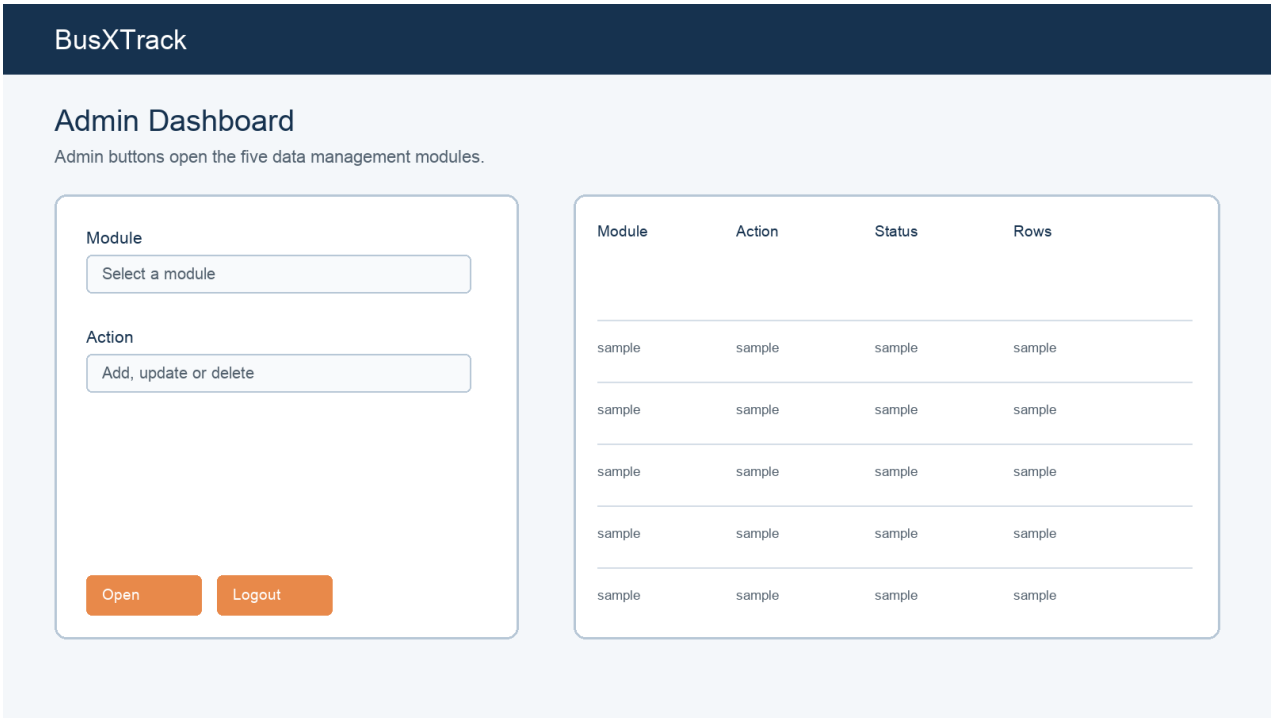


Figure 8. Admin dashboard and table interaction mockup

21 Database Table Specifications

admins stores the admin_id primary key, unique username and password. The table supports the login screen.

drivers stores driver_id, driver_name, phone_number and license_number. A bus refers to a driver through driver_id.

routes stores route_id, route_name, source and destination. Stops and buses refer to a route through route_id.

stops stores an ordered stop list. It contains route_id, stop_name, stop_order and minutes_from_start.

buses stores bus_number and capacity along with route_id and driver_id. timetable stores departures and trip_type for a bus and route.

The separation of entities prevents update anomalies. Changing a driver's phone number happens in the drivers table, not in every bus or timetable row. Adding a new stop adds a row in stops without changing the route definition. These are simple examples of why a relational database is useful.

Entity	Example row	Used by
drivers	Ramesh Kumar	Bus assignment and details
routes	Route A	Stops and buses
stops	Market Road / 15 min	Stop search and ETA
buses	BX101 / capacity 45	Bus search
timetable	08:00 / MORNING	ETA and status

22 Keys Constraints and Relationships

Every table has an integer primary key generated with AUTO_INCREMENT. This gives each row a stable identifier even when names change.

The bus_number and admin username are unique. Required columns use NOT NULL so incomplete records are rejected at the database level.

A stop cannot exist without a route. A bus cannot be assigned without an existing route and driver. A timetable row refers to an existing bus and route.

The schema uses foreign keys to express these relationships. ON DELETE CASCADE is used where dependent rows should be removed with their parent, such as timetable rows when a bus is deleted.

The keys also support the GUI combo boxes. The user sees a route name or driver name, but the ComboItem stores the related integer id. The DAO sends those ids to MySQL, which preserves the relationship even if the displayed name is edited later.

23 SQL and JDBC Flow

The SQL schema first creates the busxtrack database and then creates tables in an order that respects foreign-key dependencies. Sample rows are inserted for the demonstration.

When the user searches a bus, BusDAO runs a query using the entered bus number. The returned Bus object provides ids used by the other DAO classes to load route, driver, stops and timetable data.

JDBC uses DriverManager and the MySQL Connector/J driver. The DBConnection class stores the local URL, username and password in one place so the student can update credentials.

The SQL_Sample_Queries.sql file contains joins that can be shown in MySQL Workbench during the database portion of the examination.

The SQL script is both a setup file and a demonstration artifact. It creates the database, creates the tables, inserts the sample data and can be rerun when a clean demonstration database is needed. The separate query appendix gives the evaluator readable examples of joins and calculations.

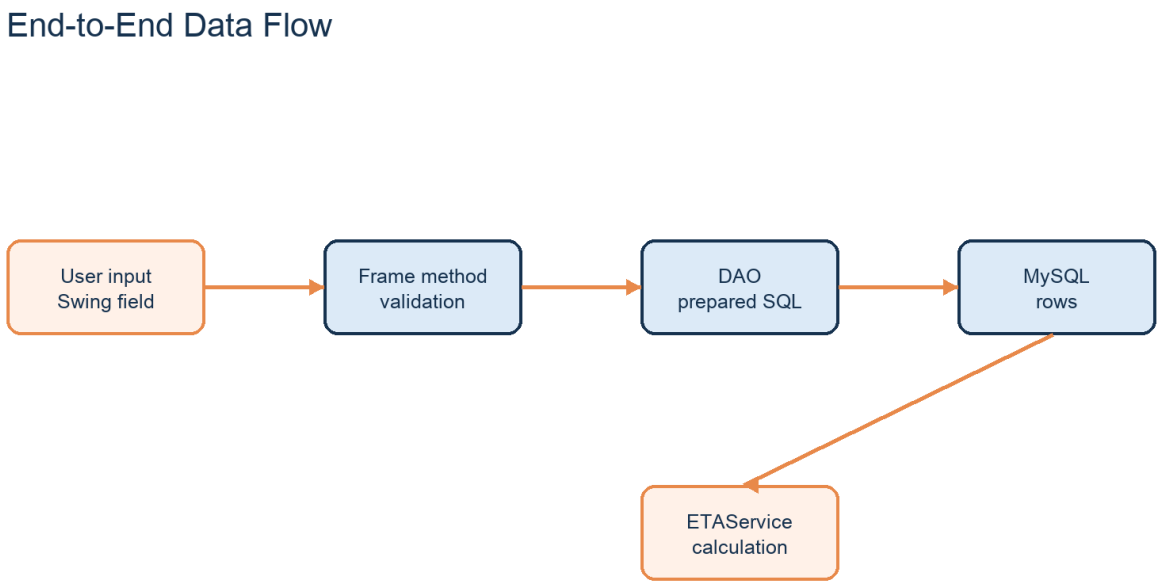


Figure 9. Data flow from a Swing input to a MySQL result

24 ETA Algorithm Explanation

ETAService reads LocalTime.now() every time a search result is calculated. The departure time is read from a java.sql.Time value returned from the timetable table.

For each stop, the service adds minutes_from_start to the departure time. If the current time is before this expected time, it calculates the minute difference and displays Arrives in a number of minutes.

If the expected time has just passed, the service displays Arriving now for a short two-minute window. Otherwise it displays Already passed.

The overall bus status compares the current time with the departure and the last configured stop. This gives NOT STARTED, RUNNING or COMPLETED.

The algorithm is deterministic for a particular computer time and database schedule. Two computers with different clock times may show different ETA text, which is expected for a simulation. The team should explain that the time input is automatic and that the schedule data is the part entered by the administrator.

25 Testing Strategy

Testing focuses on the actions that can be demonstrated in the GUI. Each management form is checked for empty input, successful insert, table refresh, row selection, update and delete.

Student search is checked with a valid bus number, a missing bus number, a valid stop name, a missing stop name and a stop with no match.

Database testing checks that the SQL file creates the schema, loads the sample rows and enforces the main key relationships.

Time testing uses timetable examples before departure, during the route and after the last stop. The expected result is compared with the ETAService output.

Testing is appropriate for the risk of a classroom prototype. The team does not need a large automated testing framework. It needs repeatable manual cases that prove the main paths: start, login, CRUD, search, invalid input, database connection and time calculation.

Area	Input	Expected evidence
Startup	Run Main	Login screen opens
Student	BX101	Bus result table loads
Stop	Market Road	Matching route and ETA appear
Admin	admin/admin123	Dashboard opens
Validation	Blank field	Message dialog appears

26 Test Cases and Expected Results

TC01: Launch Main. Expected: LoginFrame opens.

TC02: Click Continue as Student. Expected: StudentHomeFrame opens.

TC03: Search BX101. Expected: route, driver, timetable, status and ETA rows appear.

TC04: Search Market Road. Expected: matching bus and expected arrival rows appear.

TC05: Submit an empty search. Expected: a validation message appears.

TC06: Login with admin/admin123. Expected: AdminDashboardFrame opens.

TC07: Add a driver with a blank field. Expected: the record is rejected with a message.

TC08: Enter an invalid timetable time. Expected: the time format message appears.

The test cases can be used as a live checklist. The student can demonstrate TC01, TC02, TC03 and TC04 in the GUI, show TC06 for the admin panel and explain TC08 using the timetable form. The remaining cases can be described from the validation code and SQL constraints.

Test ID	Scenario	Expected result	Status
TC01	Start application	LoginFrame	Pass
TC02	Student entry	StudentHomeFrame	Pass
TC03	Search BX101	Schedule and ETA	Pass
TC04	Search Market Road	Stop result	Pass
TC05	Blank search	Validation dialog	Pass
TC06	Admin login	Dashboard	Pass

27 Error Handling and Risks

The main external dependency is the local MySQL server. If the server is stopped or the password is wrong, the application shows a database error and the student should check DBConnection.java.

Foreign-key errors can happen if a user deletes a driver or route that is still referenced. The schema prevents invalid references, and the UI displays the SQL error message.

A timetable is only a prediction based on configured minutes. Traffic, delays and a bus taking a different route are outside the current model.

The project risk is controlled by keeping a sample dataset in busxtrack.sql and by demonstrating the preloaded records before attempting live editing.

A practical risk is changing the sample records before the examination. The safest approach is to keep the original SQL script and run it on a fresh database if necessary. The team should also avoid deleting the sample driver, route or bus before demonstrating the student screens.

28 Project Management and Contributions

The work is divided into requirements and documentation, database design, Java model and DAO classes, Swing screens, ETA logic and testing. Every group member should be able to explain the complete workflow, even if individual implementation tasks were divided.

A practical presentation split is: one member explains the problem and diagrams, one explains the Swing screens and navigation, one explains MySQL and SQL queries, and all members explain the ETA formula and limitations.

The project plan moves from schema design to models, JDBC, GUI, integration and testing. The current prototype has reached the documentation and mid-term demonstration stage.

The team contribution discussion should remain accurate. Each member can claim a module they understand, but every member should be able to explain the full flow from LoginFrame to MySQL and ETAService. The report provides a common vocabulary so the group does not give contradictory answers.

29 Future Enhancements and Final Learning

A future version could receive GPS coordinates from a device attached to each bus and replace the timetable-only estimate with a live position. It could also add a web or mobile interface and student notifications.

Other improvements include password hashing, stronger input validation, better handling of overnight schedules, route delay reporting and an online database server.

The main learning outcome is integration. A Swing text field becomes a method argument, a DAO converts it to a prepared SQL query, MySQL returns relational data, and ETAService converts the schedule into a result that the student can understand.

This completes the design report for BusXTrack and provides a realistic base for the next semester phase.

The current design leaves a clear extension point. A future location service could provide a bus position to a new service class while leaving the student screens and database relationships largely similar. This is why documenting the present limitation is more useful than describing the prototype as live tracking.